QA Final Project Group Report

Bryson LeBlanc
Cole Suchan
Helen Grogan

Task 1

Generating requirements was primarily data manipulation in its simplest form - sorting through markdown and json files to parse into a desired format based on some rules. It did familiarize us with some neat parser argument options that make running the scripts easy without a DB.

```python
# ---------- Arguments ----------
parser = argparse.ArgumentParser(description="Generate requirement JSON from CFR Markdown")
parser.add_argument("--input", "-i", required=True, help="Input Markdown file (.md)")
parser.add_argument("--output", "-o", required=True, help="Output JSON file")
parser.add_argument("--structure", "-s", required=True, help="Output structured JSON file")
parser.add_argument("--cfr", "-c", required=True, help="CFR section (e.g., 21 CFR 117.130)")
args = parser.parse_args()

INPUT_MD = args.input
OUTPUT_REQUIREMENTS_JSON = args.output
OUTPUT_STRUCTURED_JSON = args.structure
CFR_SECTION = args.cfr
```

It was also nice to learn how to implement a python debugger through the `launch.json`.

```json
{
    "name": "Python Debugger: generate requirements script",
    "type": "debugpy",
    "request": "launch",
    "program": "${workspaceFolder}/scripts/generate_requirements.py",
    "console": "integratedTerminal",
    "args": [
        "--input", "Input CFR File/CFR-117.130.md",
        "--output", "real outputs/requirements.json",
        "--structure", "real outputs/expected_structure.json",
        "--cfr", "21 CFR 117.130"
    ]
},
```

This made testing as easy as selecting the right debugger and pressing “Run”.

Task 2

Generating test cases was a similar workflow to generating requirements. We did learn some neat python syntax for inline for loops and boolean expressions that tightened up our code -

```
# Find the requirement details from the requirements list
req_details = next((req for req in requirements if req["requirement_id"] == es + req_id), None)
if req_details:
```

Writing the json objects to be sent to `test_cases.json` included some string manipulation as well. Matching atomic rules with their parent requirements gave us a good hands on working example of hierarchy in quality assurance and requirements writing. It puts into perspective how unorganized things can become if not managed properly.

Task 3

Task 3 was about making sure our requirements and test cases were consistent with each other using verification and validation scripts. The goal was to catch coverage gaps and see if anything did not match structurally, then to put these checks into our CI pipeline so that they run automatically on every push.

We wrote verification.py to load both requirements.json and test_cases.json, pull out all the requirement IDs, and compare them. If any requirements did not have a matching test case, the script printed them out and exited with an error code so the CI build would fail.

```
# pull out ids from each file for comparison
tested_ids = {tc["requirement_id"] for tc in test_cases}
req_ids = {req["requirement_id"] for req in requirements}

# find which reqs have test case or not
missing = req_ids - tested_ids
found = req_ids & tested_ids
```

Validation.py rebuilt the parent-child structure from requirements.json and compared it against expected_structure.json. An entry that was missing or did not belong got flagged so that we could see exactly where the mismatch was.

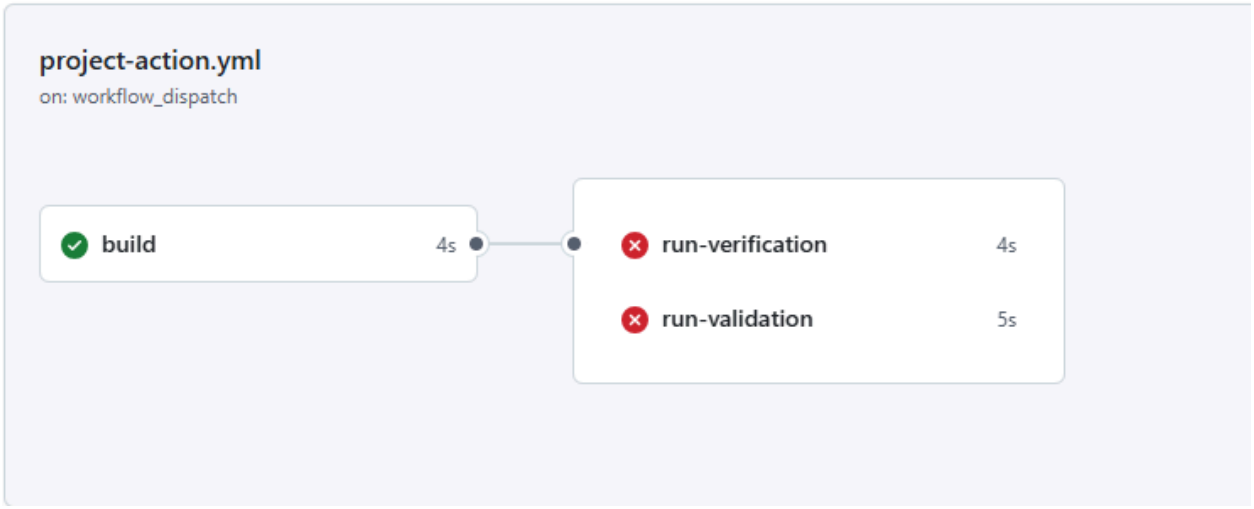
```
# check for anything missing or extra compared to expected_structure.json
errors = []
for parent, expected_children in expected.items():
    actual_children = actual.get(parent, [])
    for child in expected_children:
        if child not in actual_children:
            errors.append(f"  MISSING: {parent}{child} not found in requirements")
    for child in actual_children:
        if child not in expected_children:
            errors.append(f"  EXTRA:   {parent}{child} not in expected structure")
```

We added both scripts as separate jobs in the CI workflow that run after the build step. This means any push with mismatched files automatically gets caught without anyone having to run the scripts manually.

```
- name: Run Verification
  run: python scripts/verification.py -r "real outputs/requirements.json" -t "real outputs/test_cases.json"
```

```
- name: Run Validation
  run: python scripts/validation.py -r "real outputs/requirements.json" -e "real outputs/expected_structure.json"
```

When we first pushed, both jobs failed because requirements.json had all 26 requirements from the full CFR file while test_cases.json only covered the 10 requirements we selected. The failure output listed every missing requirement, making fixing the error easy.



After trimming requirements.json down to 1o rules and updating expected_structure.json to match, both jobs passed. This task taught us how CI can catch inconsistencies automatically and showed us how writing scripts that exit with error codes gives the pipeline the ability to stop a bad build from going through.

Task 4

Task 4 asked that we integrate forensic analysis into the verification and validation scripts as well as the CI workflow. The purpose of this task is essentially to transform volatile script outputs into a more permanent audit trail, ensuring traceability of successes and failures throughout the entire workflow.

We went about this by first taking the logging script from a previous workshop and repurposing it for this project:

```
current_dir = os.path.dirname(os.path.abspath(__file__))
file_path = os.path.join(current_dir, 'PROJECT-LOGGER.log')

format_str = '%(asctime)s:%(message)s'
#file_name  = 'PROJECT-LOGGER.log'
### creates one Global logger: logging.basicConfig
logging.basicConfig(format=format_str, filename=file_path, level=logging.INFO, force=True)
loggerObj = logging.getLogger('simple-logger')
```

This would allow us to log and track the status/output of various important checkpoints/events within our program.

```
import myLogger

logger = myLogger.giveMeLoggingObject() # logging for forensics
```

Some logs simply traced the inputs at the start of verification.py and validation.py to record exactly which version of the requirements were used.

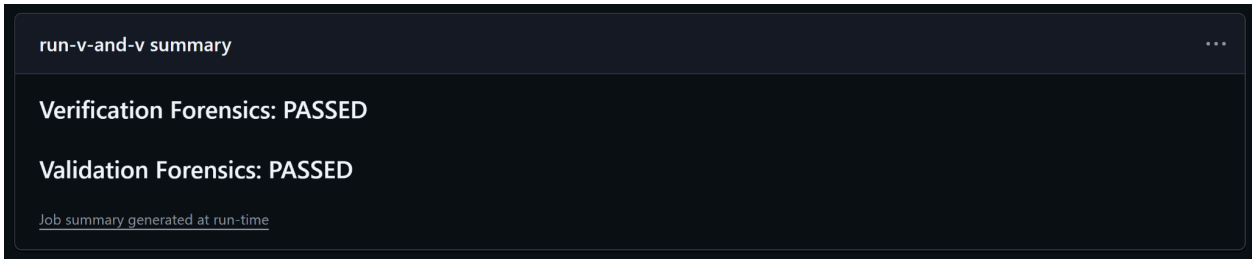
```
# LOG 1: tracing inputs (checkpoint)
logger.info(f"FORENSIC ALERT: Starting verification. \nSource: {args.requirements}, \nTests: {args.test_cases}")
```

```
# LOG 3: tracing inputs (checkpoint)
logger.info(f"FORENSIC ALERT: Starting validation. \nSource: {args.requirements}, \nExpected Structure: {args.expected}")
```

Others documented the specific structural mismatches found during verification and validation and provided a record of whether the project met its compliance criteria (PASSED) or violated them (FAILED).

```
if missing:
    # LOG 2: requirement skipped/missing (detection)
    logger.warning(f"FORENSIC ALERT: {len(missing)} requirements are missing test coverage")
    for m in sorted(missing):
        logger.info(f"Missing ID: {m}") # trace each req missing test cases
    print("\nverification FAILED")
    logger.error("FORENSIC ALERT: Verification Result: FAILED")
    sys.exit(1)
else:
    logger.info("FORENSIC ALERT: All requirements have test cases")
    logger.info("FORENSIC ALERT: Verification Result: PASSED")
    print("\nverification PASSED")
```

Finally, we utilized Github Step summaries within .github/workflows, which will generate reports that allow us to easily view the CI workflow and quickly understand why a run may have failed (immediate forensic visibility).



We also implemented artifact archiving:

```
- name: Upload Forensic Logs
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: project-forensics
    path: scripts/PROJECT-LOGGER.log
```

This process ensures that the detailed audit logs created during verification and validation are preserved as immutable artifacts.

```
2026-04-20 06:53:59,434:FORENSIC ALERT: Starting verification.
Source: real outputs/requirements.json,
Tests: real outputs/test_cases.json
2026-04-20 06:53:59,434:FORENSIC ALERT: All requirements have test cases
2026-04-20 06:53:59,434:FORENSIC ALERT: Verification Result: PASSED
2026-04-20 06:53:59,509:FORENSIC ALERT: Starting validation.
Source: real outputs/requirements.json,
Expected Structure: real outputs/expected_structure.json
2026-04-20 06:53:59,509:FORENSIC ALERT: Validation Result: PASSED
```

The process of implementing forensic analysis into our project, we reinforced our knowledge of the logging process. We essentially learned how to create a "black box" for software development that displays/proves what happened during a build.

We also learned how to use CI (GitHub Actions) as additional security. By writing scripts that return error codes, we saw how a pipeline can automatically stop a project from moving forward if it doesn't meet strict compliance.